APPENDIX A
THEORETICAL PROOFS

A. Proof of Theorem 1

Proof. We consider standard gradient updates:

$$\begin{aligned}\mathbf{W}_{t+1}^{\mathcal{T}} &= \mathbf{W}_t^{\mathcal{T}} - \eta\nabla\mathcal{L}^{\mathcal{T}}(\mathbf{W}_t^{\mathcal{T}}),\\ \mathbf{W}_{t+1}^{\mathcal{S}} &= \mathbf{W}_t^{\mathcal{S}} - \eta\nabla\mathcal{L}^{\mathcal{S}}(\mathbf{W}_t^{\mathcal{S}}).\end{aligned} \tag{30}$$

Then, the difference between the two trajectories at step $t+1$ can be calculated as:

$$\begin{aligned}\mathbf{W}_{t+1}^{\mathcal{S}} - \mathbf{W}_{t+1}^{\mathcal{T}} &= \left(\mathbf{W}_t^{\mathcal{S}} - \eta\nabla\mathcal{L}^{\mathcal{S}}(\mathbf{W}_t^{\mathcal{S}})\right) - \left(\mathbf{W}_t^{\mathcal{T}} - \eta\nabla\mathcal{L}^{\mathcal{T}}(\mathbf{W}_t^{\mathcal{T}})\right)\\ &= \left(\mathbf{W}_t^{\mathcal{S}} - \mathbf{W}_t^{\mathcal{T}}\right) - \eta\left(\nabla\mathcal{L}^{\mathcal{S}}(\mathbf{W}_t^{\mathcal{S}}) - \nabla\mathcal{L}^{\mathcal{T}}(\mathbf{W}_t^{\mathcal{T}})\right).\end{aligned} \tag{31}$$

Taking the norm, we have:

$$\left\|\mathbf{W}_{t+1}^{\mathcal{S}} - \mathbf{W}_{k+1}^{\mathcal{T}}\right\| \le \left\|\mathbf{W}_t^{\mathcal{S}} - \mathbf{W}_t^{\mathcal{T}}\right\| + \eta\left\|\nabla\mathcal{L}^{\mathcal{S}}(\mathbf{W}_t^{\mathcal{S}}) - \nabla\mathcal{L}^{\mathcal{T}}(\mathbf{W}_t^{\mathcal{T}})\right\|. \tag{32}$$

Finally, we apply Eq. (32) recursively from $t=0$ to $t=T-1$, noting that $\mathbf{W}_0^{\mathcal{S}} = \mathbf{W}_0^{\mathcal{T}}$, we obtain:

$$\left\|\mathbf{W}_T^{\mathcal{S}} - \mathbf{W}_T^{\mathcal{T}}\right\| \le \eta\sum_{k=0}^{T-1}\left\|\nabla\mathcal{L}^{\mathcal{S}}(\mathbf{W}_t^{\mathcal{S}}) - \nabla\mathcal{L}^{\mathcal{T}}(\mathbf{W}_t^{\mathcal{T}})\right\|. \tag{33}$$

□

B. Proof of Theorem 2

Proof. As the gradient matching objective is commonly calculated for each class separately, we define the class-wise loss for $\mathcal{T}$ and $\mathcal{S}$ as:

$$\begin{aligned}\mathcal{L}_{\mathrm{GM}} &= \sum_{i=0}^{n-1}\left\|\frac{1}{|n_i|}\nabla\mathcal{L}_i^{\mathcal{T}}(\mathbf{W}) - \frac{1}{|n_i'|}\nabla\mathcal{L}_i^{\mathcal{S}}(\mathbf{W})\right\|^2\\ &= \sum_{i=0}^{C-1}\left\|\frac{1}{|n_i|}(\Phi_i^\top\Phi_i\mathbf{W} - \Phi_i^\top Y_i) - \frac{1}{|n_i'|}({\Phi_i'}^\top\Phi_i'\mathbf{W} - {\Phi_i'}^\top Y_i')\right\|^2\\ &= \sum_{i=0}^{C-1}\left\|\left(\tfrac{1}{|n_i|}\Phi_i^\top\Phi_i - \tfrac{1}{|n_i'|}{\Phi_i'}^\top\Phi_i'\right)\mathbf{W} - \left(\tfrac{1}{|n_i|}\Phi_i^\top Y_i - \tfrac{1}{|n_i'|}{\Phi_i'}^\top Y_i'\right)\right\|^2\\ &\le \sum_{i=0}^{C-1}\left\|\tfrac{1}{|n_i|}\Phi_i^\top Y_i - \tfrac{1}{|n_i'|}{\Phi_i'}^\top Y_i'\right\|^2 + \sum_{i=0}^{C-1}\left\|\tfrac{1}{|n_i|}\Phi_i^\top\Phi_i - \tfrac{1}{|n_i'|}{\Phi_i'}^\top\Phi_i'\right\|^2\|\mathbf{W}\|^2\\ &= \left\|\mathbf{P}\Phi(X) - \mathbf{P}'\Phi(X')\right\|^2 + \sum_{i=0}^{C-1}\left\|\tfrac{1}{|n_i|}\Phi_i^\top\Phi_i - \tfrac{1}{|n_i'|}{\Phi_i'}^\top\Phi_i'\right\|^2\|\mathbf{W}\|^2.\end{aligned} \tag{34}$$

This decomposition reveals that distribution matching, when combined with second-order embedding alignment, provides an upper bound for class-wise gradient matching. □

C. Proof of Theorem 3

Proof. Recall that MCKP is as follows: Given m mutually disjointed groups, where group G_i contains a set of K_i items $\{x_{i,1}, x_{i,2}, \ldots, x_{i,K_i}\}$. Each item $x_{i,k} \in G_i$ has a value $v_{i,k} \in \mathbb{R}_{\ge 0}$ and a weight $w_{i,k} \in \mathbb{Z}_{>0}$. The capacity of the knapsack is W. MCKP aims to select exactly one item from each group such that the total weight $\sum_i w_{i,\sigma(i)} \le W$ is within the budget and the total value $\sum_i v_{i,\sigma(i)}$ is maximized. Here, the mapping $\sigma : \{1,2,...,m\} \to \mathbb{Z}_{>0}$ specifies the index of the selected item in each group, i.e., $x_{i,\sigma(i)} \in G_i$ denotes the item chosen for group i. Based on this, we construct an instance of class-adaptive cluster allocation as: there are m classes of samples which corresponds to the number of groups in MCKP. The candidate set of cluster numbers for class i is denoted as $\{n'_{i,1}, n'_{i,2}, \cdots, n'_{i,K_i}\}$, where $n'_{i,k} = k$ and $K_i = \max\{N' - (C-1),\, n_i\}$. This defines the discrete search space for each class i, which is structurally equivalent to the item set G_i in MCKP. Each element in this set corresponds to an item in MCKP, where the sample count $n'_{i,k}$ also plays a role of the item's weight $w_{i,k}$. Assume that we enumerate all candidate $n'_{i,k}$, and compute corresponding optimal clustering loss $\mathcal{L}(\{S_j^{i,k}\}^*)$, which corresponds to the negative value $-v_{i,k}$ in MCKP. The total cluster budget is set to W, satisfying that $\sum_i w_{i,\sigma(i)} = W$. From this instance, we can derive a perfect matching for the MCKP problem. This completes the NP-hardness proof. □

D. Proof of Theorem 4

Proof. In Algorithm 1, the initialization stage incurs its main cost in the ClassWiseClustering (Algorithm 2), invoked at line 7 of Algorithm 1, where each class i performs RunK-Means on $X_i \in \mathbb{R}^{n_i \times F}$ for I iterations. This step requires $\mathcal{O}(n_i n_i' IF)$ time, where $n_i' = \lfloor n_i \cdot r \rfloor$. Hence, the overall initialization complexity is $\mathcal{O}(IF\sum_{i=0}^{C-1} n_i \lfloor n_i r \rfloor)$. After initialization, the algorithm iteratively alternates between SoftAllocate (Algorithm 3, invoked at line 12 of Algorithm 1), which runs in constant time, and ClassWiseClustering (line 13), which dominates the iteration cost. In the worst case, clustering must be recomputed for all classes in each iteration. Let $\bar{n}' = \frac{1}{C}\sum_{i=0}^{C-1} n_i' = \frac{N'}{C}$ denote the average number of clusters per class. Then, over T iterations, the total cost of the search phase is $\mathcal{O}(T\sum_{i=0}^{C-1} n_i \bar{n}_i IF) = \mathcal{O}(T \cdot \frac{NN'IF}{C})$. After convergence, Algorithm 1 performs RunKMeans(X_i, n_i') for each class (line 27) to obtain the final partitions $\{\{S_j^i\}^*\}$. This post-processing cost is negligible when T is large. The time complexity of the search over T steps is $\mathcal{O}(T \cdot \frac{NN'IF}{C})$. Overall, the time complexity of HFILS is $\mathcal{O}(IF\sum_{i=0}^{C-1} n_i \cdot \lfloor n_i \cdot r \rfloor + T \cdot \frac{NN'IF}{C}) = \mathcal{O}(T \cdot \frac{NN'IF}{C})$. □

APPENDIX B
RELATED WORK

A. Coreset selection

Coreset selection methods aim to select a representative subset that achieves comparable model performance to training on the full dataset. Existing methods primarily rely on intermediate objectives that enforce distributional consistency between the subset and the original dataset [1]–[5]. Representative approaches include herding [1], which greedily adds samples to minimize the feature-space distance between the centers of the subset and the full dataset, and k-center [4], which iteratively selects samples to maximize spatial coverage. More recently, some methods have leveraged training signals such as gradients, loss values, or forgetting events to guide the selection of informative samples [6]–[10]. Despite their effectiveness, coreset selection methods share inherent limitations: since they can only select a small subset of existing samples, they are constrained by the original dataset and cannot generate new, potentially more representative instances [11]–[13]. Therefore, they fail to fully exploit the rich information in the full dataset, inevitably leading to substantial information loss [7]. This problem is further amplified when informative

signals are broadly distributed rather than concentrated in a few samples [6].

APPENDIX C
COMPARISON WITH CGC

A. Difference in problem definition

The underlying problem definition and structural assumptions in our work differ fundamentally from those in [14]. Specifically, [14] focuses on graph data condensation, aiming to distill a large-scale graph into a compact yet informative one while preserving training utility for deep graph models. Accordingly, structural dependencies among entities are explicitly modeled by the adjacency matrix A and retained in the condensed graph. In contrast, our formulation targets tabular data composed of independent and identically distributed samples with heterogeneous feature types. We explicitly distinguish numerical and categorical attributes, denoted as $x_{i,j}^{num}$ and $x_{i,j}^{cat}$, respectively, and aim to condense feature-heterogeneous tabular samples for efficient training of deep tabular models without modeling any inter-sample structural information.

B. Difference in theoretical framework

Our theoretical analysis process for establishing a unified condensation framework differs from [14]. [14] unifies performance matching, parameter matching, and distribution matching in graph condensation, treating gradient matching as a representative instance of parameter matching and establishing DM as a proxy for PM. In contrast, we focus on independent-sample condensation and provide a finer-grained analysis of parameter matching by explicitly distinguishing gradient matching and training trajectory matching. We show that step-wise gradient discrepancies upper-bound long-range trajectory misalignment and further prove that DM upper-bounds GM, thereby yielding a unified framework encompassing MTT, GM, and DM.

C. Difference in label allocation strategies

Our class partitioning objective and optimization space differ from [14]. [14] focuses on graph condensation and performs partitioning on structure-aware node representations obtained via linear message passing. In contrast, we target tabular data condensation and, under a linearization simplification, conduct partitioning directly in the original sample space, since no structural dependencies exist among samples. Moreover, [14] considers only within-class partitioning with a fixed number of condensed samples per class, whereas we jointly optimize inter-class label allocation and intra-class clustering, leading to the Class-Adaptive Cluster Allocation Problem (CCAP), which adaptively determines the number of condensed samples per class.

APPENDIX D
AUTOENCODER FOR STRING-TYPE FEATURE ENCODING

A. Computational Cost Analysis

To assess the computational efficiency of the autoencoder module, we empirically evaluate its runtime on three representative tabular datasets with string-type categorical features, namely Adult (AD), Diabetes130US (DA), and Airlines (AI). The results show that the encoding process is highly efficient. In practice, the model converges within a small number of epochs and is therefore trained for a fixed 10 epochs as an offline preprocessing step. Under this setting, the runtime is 6.8 seconds on AD, 46.3 seconds on DA, and 98.5 seconds on AI. Overall, the proposed autoencoder serves as a lightweight and efficient preprocessing component that is trained only once per dataset, incurring a low computational cost that is negligible compared to the training overhead of prior learning-based condensation methods such as GM and DM.

APPENDIX E
OUR APPROACH

A. Class-wise clustering algorithm

Algorithm 2 illustrates the CW-Clus procedure. For each class, the clustering loss is either computed via K-means (lines 3–9) or retrieved from the cached loss matrix $\mathbf{M}$ when the corresponding entry already exists (i.e., the condition in line 4 is not met). The total clustering loss $\mathcal{L}_c$ is then accumulated over all classes (line 10) and returned as the overall objective value (line 12).

Algorithm 2 ClassWiseClustering

Require: Original dataset $\mathcal{T} = (X, Y)$, number of classes C, current cluster allocation $\{n'_i\}$, total cluster count N', adaptive exponent γ, loss matrix $\mathbf{M}$
Ensure: Total clustering loss $\mathcal{L}_c$
1: $X_i \leftarrow \{x \in X \mid y(x) = i\}$ for each $i \in \{0, 1, \ldots, C-1\}$
2: $\mathcal{L}_c \leftarrow 0$
3: **for** $i \in \{0, 1, \ldots, C-1\}$ **do**
4: **if** $\mathbf{M}[i, n'_i] = \infty$ **then**
// COMPUTE CLASS-WISE CLUSTERING LOSS
5: SSD, $\{S_j^i\}^* \leftarrow$ RunKMeans(X_i, n'_i)
6: $n_i \leftarrow |X_i|$
// COMPUTE CLASS-WISE SCALE FACTOR
7: $w \leftarrow \frac{1}{n_i^\gamma}$
8: $\mathbf{M}[i, n'_i] \leftarrow$ SSD $\times\, w$
9: **end if**
10: $\mathcal{L}_c \leftarrow \mathcal{L}_c + \mathbf{M}[i, n'_i]$
11: **end for**
12: **return** $\mathcal{L}_c$

B. Soft allocate algorithm

To facilitate dynamic local search and better approximate the optimum of in Eq. (12), we design a neighborhood scanning strategy based on composite reallocation moves. In each move, the algorithm decreases the cluster count of one source class while simultaneously increasing those of multiple target classes, thereby realizing adaptive redistribution across classes. Instead of a rigid one-to-one adjustment where a single class gains clusters strictly at the expense of another, our approach employs a *softer reallocation mechanism* that enables smoother, more flexible, and better-balanced redistribution across multiple classes.

The soft allocation procedure is illustrated in Algorithm 3. It first selects a source class c_{src} with more than one cluster (line 1) and constructs the target class set $\mathcal{C}_{\text{tgt}}$, which contains classes that can still receive additional clusters within

their upper limits (lines 2–5). A real step size s is then randomly sampled from $[1, s_{\max}]$ and bounded by the transferable capacity of c_{src} (lines 6–7), introducing controlled stochasticity that promotes diverse yet feasible adjustments. After subtracting s clusters from the source class (line 8), the reallocation proceeds according to the remaining capacities of target classes. For each class in $\mathcal{C}_{\text{tgt}}$, the algorithm estimates its residual capacity r_c (lines 9–11) and determines a provisional allocation a_c proportional to its relative ratio $\frac{r_c}{r_{\text{tot}}}$ (lines 13–14), guided by the elbow heuristic that prioritizes classes expected to yield higher clustering gains. To handle rounding effects, the remaining clusters δ are sequentially assigned to classes with larger residual capacities until all clusters are fully allocated (lines 16–25). Finally, the per-class cluster counts n'_c are updated (lines 26–28) to reflect the completed redistribution, resulting in an updated allocation $\{n'_i\}$ (line 29).

Algorithm 3 SoftAllocate

Require: Per-class sample sizes $\{n_i\}$, current cluster allocation $\{n'_i\}$, number of classes C, total cluster count N', current max step size $s_{\max}$
Ensure: Updated per-class cluster sizes $\{n'_i\}$
1: $c_{\text{src}} \sim \text{Uniform}(\{\, i \mid n'_i > 1 \,\})$
2: **for** $i \in \{0, 1, \ldots, C-1\}$ **do**
3: $\quad n'_{i,\max} \leftarrow \min\{N' - (C-1),\, n_i\}$
4: **end for**
5: $\mathcal{C}_{\text{tgt}} \leftarrow \{\, i \mid n'_i < n'_{i,\max} \wedge i \neq c_{\text{src}} \,\}$
$\quad$ // RANDOMIZE A REAL STEP SIZE
6: $s \sim \text{Uniform}(\{1, 2, \ldots, s_{\max}\})$
7: $s \leftarrow \min(s,\ n'_{c_{\text{src}}} - 1)$
8: $n'_{c_{\text{src}}} \leftarrow n'_{c_{\text{src}}} - s$
$\quad$ // ALLOCATE BY REMAINING CAPACITY RATIO
9: **for all** $c \in \mathcal{C}_{\text{tgt}}$ **do**
10: $\quad r_c \leftarrow n'_{c,\max} - n'_c$
11: **end for**
12: $r_{\text{tot}} \leftarrow \sum_{c \in \mathcal{C}_{\text{tgt}}} r_c$
13: **for all** $c \in \mathcal{C}_{\text{tgt}}$ **do**
14: $\quad a_c \leftarrow \left\lfloor s \cdot \dfrac{r_c}{r_{\text{tot}}} \right\rfloor$
15: **end for**
$\quad$ // DISTRIBUTE UNASSIGNED CLUSTERS
16: $\delta \leftarrow s - \sum_{c \in \mathcal{C}_{\text{tgt}}} a_c$
17: Sort $\mathcal{C}_{\text{tgt}}$ by descending r_c
18: **for all** $c \in \mathcal{C}_{\text{tgt}}$ **do**
19: $\quad$ **if** $\delta = 0$ **then**
20: $\quad\quad$ **break**
21: $\quad$ **end if**
22: $\quad$ **if** $n'_c + a_c + 1 \leq n'_{c,\max}$ **then**
23: $\quad\quad a_c \leftarrow a_c + 1,\ \delta \leftarrow \delta - 1$
24: $\quad$ **end if**
25: **end for**
26: **for all** $c \in \mathcal{C}_{\text{tgt}}$ **do**
27: $\quad n'_c \leftarrow n'_c + a_c$
28: **end for**
29: **return** $\{n'_i\}$

APPENDIX F
MORE EXPERIMENTAL SETTINGS

A. Detailed Baselines Descriptions

We compare our method against a set of representative baselines, including coreset selection approaches (Random, Herding [1], and K-Center [4]) and dataset condensation methods (DM [11], GM [13], and MTT [12]).

- **Random.** Random selects a subset of training samples uniformly at random from the original data as the coreset.
- **Herding** [1]. Herding is a greedy algorithm that iteratively selects samples closest to the cluster center.
- **K-Center** [4]. K-Center picks multiple center points such that the largest distance between a data point and its nearest center point is minimized.
- **DM** [11]. DM learns condensed dataset by matching the feature embeddings between real and synthetic samples.
- **GM** [13]. GM proposes to infer the synthetic dataset by matching the gradients of a model trained on synthetic dataset to that on the original dataset.
- **MTT** [12]. MTT builds a condensation dataset to match the parameter trajectory of full dataset training.

B. Evaluation Metrics Details

To evaluate the classification performance of models trained on condensed datasets, we adopt two widely used metrics: *Accuracy* and *Macro-F1*. Accuracy is defined as the proportion of correctly predicted instances:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{35}$$

where TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives, respectively. To further provide a balanced view of classification performance across all classes, we utilize Macro-F1, which is the harmonic mean of precision and recall, computed independently for each class and then averaged:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=0}^{C-1} \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \tag{36}$$

where $\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}$ and $\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}$.

C. Implementation Details

The pipeline of dataset condensation has two stages: (1) **Condensation step**, where we apply the DC methods on the training set to obtain small-scale synthetic data. (2) **Evaluation step**, where we train deep models on the condensed data from scratch and then evaluate their performance on the test set of the original data. For existing DC methods (i.e., DM, GM, and MTT) that requires model training, we use FT-Transformer [15], an effective and widely adopted tabular learning method, as the relay model. Our effectiveness experiments consider three reduction rates (1%, 0.1%, and 0.01%), with 0.1% used as the default setting in subsequent experiments unless stated otherwise. For our approach, we set the maximum iteration to $T = 1000$, the tolerance to $\epsilon = 0.01$, and the early-stopping patience to $p = 10$. The adaptive weighting exponent γ is tuned over $\{0.25, 0.5, 0.75, 1.0\}$. Additionally, the step size scaling factor l is varied within $\{0.1, 0.3, 0.5, 0.7, 0.9\}$, while the cluster reweighting factor ρ is selected from $\{0.25, 0.5, 0.75, 1.0\}$. For all baselines, we report results using the official implementations with default hyperparameters specified in the original papers, and apply label encoding for categorical features. Methods that exceed the runtime limit (24 hours during condensation, or 5 days for expert trajectories generation during pre-processing in MTT) are denoted as OOT (out of time), while those that fail due

to memory allocation errors are denoted as OOM (out of memory). The experiments are conducted on a Linux server with an Intel(R) Xeon(R) Gold 6330 CPU, a NVIDIA A800 GPU, and 128GB of RAM. To eliminate randomness, each experiment is repeated five times, and the average results with standard deviation are reported.

APPENDIX G
ADDITIONAL EXPERIMENTAL RESULTS

A. Additional Performance Analysis

We observe that on MI, C^{2}TC already recovers over 93% accuracy of the full-data performance at $r = 0.01\%$, with further increasing r to 1% yielding only marginal gains (e.g., from 53.2% to 54.4%). This behavior can be attributed to the following reasons. First, benefiting from class-adaptive clustering, C^{2}TC is able to extract highly representative samples even at extremely low reduction rates. Second, the large-scale MI dataset exhibits substantial redundancy, allowing most informative patterns to be captured at very low reduction rates and leaving limited room for further improvement. Moreover, although increasing r from 0.01% to 1% corresponds to a $100\times$ relative increase, the absolute data scale remains small, resulting in only a modest performance gain. Based on these observations, we recommend small r values (e.g., $r = 0.01\%$) when computational or memory budgets are limited, and $r = 1\%$ as a robust default when a better balance between performance and generalization is desired.

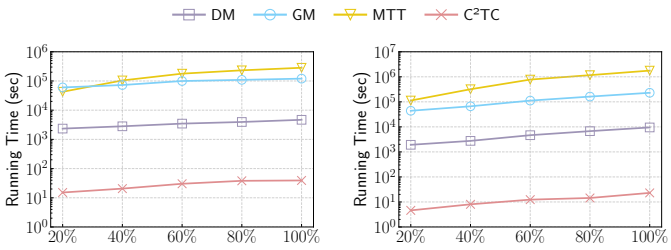
To assess the reliability of the observed improvements in accuracy (Acc) and Macro-F1 (MF1), we conduct paired t-tests [16] comparing C^{2}TC with the strongest coreset and condensation baselines, namely Herding and MTT, at $r = 0.1\%$, and report the corresponding p-values. Smaller p-values indicate stronger evidence that the observed performance differences are not due to random variation, and results with $p < 0.05$ are considered statistically significant. The tests were performed over 25 paired experimental runs per dataset, obtained from 5 independently generated synthetic datasets, each under 5 random seeds. As shown in Table A1, C^{2}TC achieves statistically significant gains ($p < 0.05$) in accuracy on 9 out of the 10 datasets against Herding and 8 out of the 9 available datasets against MTT (excluding EP due to OOT), as well as in Macro-F1 on 8 out of the 10 and 8 out of the 9 datasets, respectively. In the few cases where improvements are not statistically significant (e.g., on the JA dataset against Herding as measured by accuracy), our method remains statistically comparable to the coreset baseline while still significantly outperforming the condensation competitor (MTT, $p = 3.75 \times 10^{-3}$).

TABLE A1: Paired t-test results (p-values) comparing C^{2}TC against competitive baselines (Herding and MTT). Bold values indicate statistical significance ($p < 0.05$). Results for MTT on EP are omitted due to OOT.

Dataset	C^{2}TC vs. Herding		C^{2}TC vs. MTT	
	Acc p-val	MF1 p-val	Acc p-val	MF1 p-val
EL	$\mathbf{6.90\times10^{-33}}$	$\mathbf{7.34\times10^{-30}}$	$\mathbf{3.78\times10^{-15}}$	$\mathbf{8.79\times10^{-15}}$
AD	$\mathbf{4.01\times10^{-17}}$	$\mathbf{3.20\times10^{-20}}$	$\mathbf{5.37\times10^{-13}}$	$\mathbf{5.13\times10^{-10}}$
JA	4.76×10^{-1}	$\mathbf{1.44\times10^{-6}}$	$\mathbf{3.75\times10^{-3}}$	$\mathbf{2.48\times10^{-2}}$
DA	$\mathbf{4.45\times10^{-9}}$	$\mathbf{1.05\times10^{-13}}$	$\mathbf{9.07\times10^{-8}}$	$\mathbf{1.78\times10^{-12}}$
RS	$\mathbf{2.28\times10^{-2}}$	5.56×10^{-2}	$\mathbf{2.18\times10^{-2}}$	$\mathbf{3.60\times10^{-2}}$
EP	$\mathbf{1.31\times10^{-21}}$	$\mathbf{1.86\times10^{-19}}$	–	–
AI	$\mathbf{1.47\times10^{-32}}$	$\mathbf{8.37\times10^{-26}}$	$\mathbf{1.99\times10^{-22}}$	$\mathbf{3.74\times10^{-14}}$
CO	$\mathbf{3.07\times10^{-25}}$	$\mathbf{4.79\times10^{-20}}$	$\mathbf{3.39\times10^{-13}}$	$\mathbf{8.42\times10^{-15}}$
HI	$\mathbf{5.18\times10^{-17}}$	$\mathbf{2.11\times10^{-22}}$	$\mathbf{6.82\times10^{-10}}$	$\mathbf{2.55\times10^{-7}}$
MI	$\mathbf{2.95\times10^{-3}}$	8.84×10^{-2}	3.25×10^{-1}	5.62×10^{-2}

B. Additional Efficiency Evaluation

Condensation time under varying N and F. In this experiment, we study the scalability of condensation methods with respect to sample size N and feature dimension F on the MI and EP datasets, respectively. Specifically, on MI we vary the sample size from 20% to 80% to form new datasets under the default reduction rate (0.1%), whereas on EP we fix the sample size and vary the feature dimension from 20% to 80% under a reduction rate of 0.01%. Figure A1(a) and Figure A1(b) show the condensation time of C^{2}TC and the three DC methods with varying N and F, respectively. As observed, C^{2}TC outperforms the other algorithms by a significant margin and scales well. For example, when the percentage of N is 20% in Figure A1(a), the condensation times of DM, GM, and MTT are 2340.2 seconds, 6.0×10^4 seconds, and 4.3×10^4 seconds, respectively, whereas C^{2}TC requires only 15.2 seconds. When the percentage of N increases to 80%, C^{2}TC completes in 38.1 seconds, while DM, GM, and MTT scale up to 3943.7, 1.1×10^5, and 2.3×10^5 seconds, respectively. In Figure A1(b), we find that C^{2}TC consistently achieves at least 2 orders of magnitude speedup over existing DC methods across different ratios of F on the EP dataset. As F increases from 20% to 80%, the condensation time of C^{2}TC rises only slightly from 4.6 seconds to 14.4 seconds. In contrast, the runtime of the fastest DC baseline, DM, grows rapidly from 1920.7 seconds to 6749.5 seconds. These results highlight the superior scalability of C^{2}TC, in line with our complexity analysis.

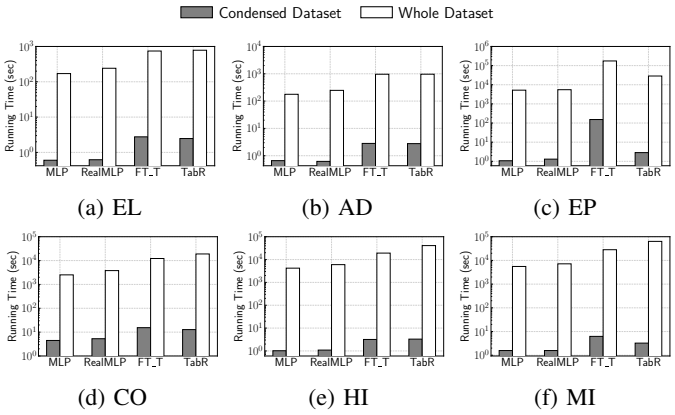


(a) MI (Vary #samples N) (b) EP (Vary #features F)

Fig. A1: Condensation time by varying N and F.

Model training time on condensed data. We further compare the training time of deep models on the condensed datasets against their original counterparts. Specifically, we evaluate four representative tabular learning architectures, namely MLP, RealMLP, FT-Transformer (FT-T), and TabR, on six datasets with different scales (EL, AD, EP, CO, HI, and MI). As shown in Figure A2, the training time on the condensed datasets is significantly reduced compared with that on the original datasets in all cases. For example, on the HI dataset, training

RealMLP on the original data takes over 5993 seconds, whereas training on the small-scale synthetic data requires only about 1 second. On the largest MI dataset, training TabR on the condensed dataset is accelerated by over 4 orders of magnitude compared with training on the original dataset, showing the efficiency and practicality of tabular dataset condensation.



(a) EL (b) AD (c) EP

(d) CO (e) HI (f) MI

Fig. A2: Training time on condensed vs. whole datasets.

C. Additional Ablation Study

Analysis of label allocation strategies. To validate the effectiveness of our adaptive allocation strategy, which is developed on top of our proposed training-free objective to appropriately determine the per-class cluster number n_i', we replace it with two static label allocation strategies, Ratio and FIPC. Ratio allocates clusters proportionally to the original class distribution, whereas FIPC assigns an equal number of synthetic samples to each class. As shown in Table A2, our class-adaptive strategy consistently achieves superior performance on both metrics across all datasets. For instance, it improves accuracy by 5.3% and Macro-F1 by 5.8% on EP, and by 4.0% and 5.9% on CO, demonstrating strong robustness across different class distribution scenarios. This is because our C^{2}TC dynamically balances the influence of majority and minority classes, leading to a better trade-off between overall accuracy and per-class performance. Compared with FIPC, Ratio achieves higher accuracy but lower Macro-F1 on imbalanced datasets (AD, CO, MI), as it allocates more clusters to majority classes and overlooks minority ones. FIPC, on the other hand, enforces uniform allocation across classes, improving minority-class performance at the cost of overall accuracy dominated by majority classes.

TABLE A2: Comparison of label allocation strategies by Accuracy (Acc) and Macro-F1 (MF1).

Strategy	EL		AD		EP		CO		MI	
	Acc	MF1	Acc	MF1	Acc	MF1	Acc	MF1	Acc	MF1
Ratio	63.0	62.9	80.4	69.7	69.9	69.0	65.6	33.1	53.5	20.9
FIPC	63.0	62.9	74.2	70.7	69.9	69.0	47.3	40.5	34.2	20.8
C^{2}TC	**64.5**	**63.5**	**81.5**	**70.8**	**75.2**	**74.8**	**69.6**	**46.4**	**53.6**	**21.5**

Analysis of categorical feature encodings. We evaluate the effectiveness of the proposed hybrid categorical feature encoding (HCFE) by comparing it with three widely adopted encoding strategies [17]: (i) label encoding, (ii) one-hot encoding followed by PCA, where PCA is applied to match feature dimensionality for fair comparison, and (iii) target encoding. All encoding strategies are evaluated on three datasets with varying scales (AD, DA, and AI). As reported in Table A3, HCFE consistently achieves the best performance across all datasets. For example, on the AD dataset, HCFE improves accuracy by 13.8% and Macro-F1 by 7.9% compared to Label Encoding, and also outperforms the most competitive baselines, including Target Encoding and One-hot + PCA, in both metrics. These results demonstrate that HCFE achieves superior accuracy and class-level balance, validating its effectiveness as an encoding strategy for condensed tabular data. Furthermore, the successful execution of the condensation process with these diverse baselines indicates that our framework is encoder-agnostic and can accommodate alternative numerical representations.

TABLE A3: Comparison of categorical feature encoding strategies for data condensation.

Dataset	Label		One-hot + PCA		Target		HCFE	
	Acc	MF1	Acc	MF1	Acc	MF1	Acc	MF1
AD	67.7	62.9	74.4	70.0	78.8	69.9	**81.5**	**70.8**
DA	50.2	24.3	51.3	30.8	47.1	33.5	**51.8**	**34.4**
AI	53.1	52.0	61.0	56.0	59.1	58.2	**61.5**	**58.4**

D. Cross-architecture Generalization

To evaluate the generalization of C^{2}TC, Table A4 reports results across six benchmark architectures for deep tabular learning: three MLP-based models (MLP, MLP_PLR [18], and RealMLP [19]) and three Transformer-based models (FT_Transformer (FT_T) [15], TabNet [20], and TabR [21]). For comparison, we also report results of Random, DM, GM, and MTT. On the high-dimensional dataset EP, existing DC methods fail due to OOM or OOT issues, so results are reported only for Random and C^{2}TC, with C^{2}TC outperforming Random by approximately 6% in terms of average accuracy. On MI, GM is also omitted due to OOT. Overall, C^{2}TC achieves the highest average accuracy across datasets, consistently benefiting a variety of deep tabular models. For instance, compared to MTT, the strongest competing DC baseline, C^{2}TC yields an average improvement of 3.1% on AD and more than 8% on EL. These results confirm the superior generalization ability of C^{2}TC, which stems from our theoretically grounded objective that ensures the quality of the condensed datasets, together with the model-free design of our algorithm that avoids architecture-specific biases and thereby enhances generalization across diverse models.

E. Parameter Analysis

Adaptive weighting exponent γ. Figure A3(a) and (b) illustrate the performance of C^{2}TC for $\gamma \in \{0.25, 0.5, 0.75, 1.0\}$. We observe that as γ increases, accuracy generally decreases while Macro-F1 improves. This trade-off arises because smaller γ values prioritize majority-class clustering, favoring

TABLE A4: Generalization of DC methods across deep tabular learning models. Avg. stands for the average test accuracy of MLP, MLP_PLR, RealMLP, FT_T, TabNet, and TabR. Results for DM, GM, and MTT on EP, and GM on MI, are omitted as they fail to scale to these datasets due to time or memory constraints. All results are reported in terms of accuracy.

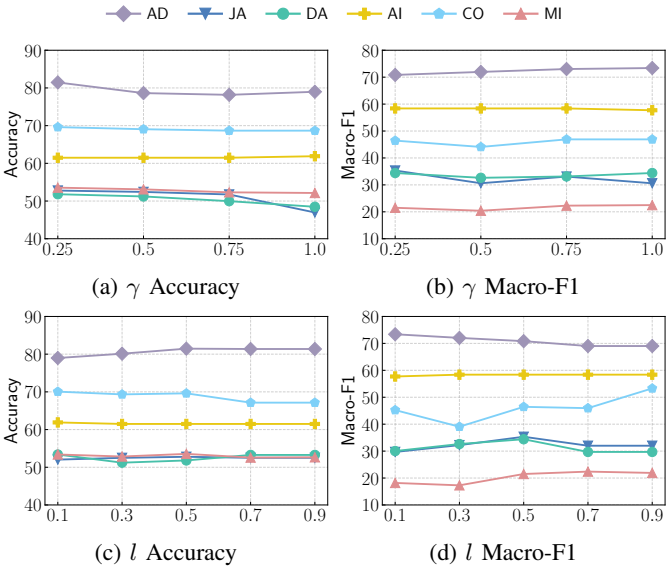
Dataset	Method	MLP-Based			Transformer-Based			Avg.
		MLP	MLP_PLR	RealMLP	FT_T	TabNet	TabR	
EL	Random	53.5	55.8	52.3	53.9	50.3	52.2	53.0
	DM	51.6	55.1	55.1	57.2	49.3	53.0	53.6
	GM	50.3	50.6	50.3	50.6	50.3	50.6	50.5
	MTT	52.6	57.0	52.7	56.0	**51.1**	53.9	53.9
	C^2TC	**64.5**	**67.5**	**62.8**	**64.5**	51.0	**61.3**	**62.0**
AD	Random	75.7	77.1	73.6	76.2	64.3	76.3	73.9
	DM	76.3	76.0	73.8	77.2	71.6	77.6	75.4
	GM	76.1	75.9	68.7	76.6	69.8	76.1	73.9
	MTT	76.2	77.3	73.9	76.8	71.1	77.7	75.5
	C^2TC	**81.5**	**80.3**	**77.8**	**79.3**	**72.5**	**79.9**	**78.6**
EP	Random	60.1	66.8	**50.4**	50.0	50.1	53.8	55.2
	C^2TC	**75.2**	**83.7**	50.2	**50.3**	**50.5**	**56.8**	**61.1**
CO	Random	63.9	67.1	52.5	61.6	56.6	69.2	61.8
	DM	61.5	56.9	**54.3**	57.7	57.3	60.0	57.9
	GM	47.9	51.9	43.8	53.1	54.9	45.6	49.5
	MTT	66.7	68.1	53.5	63.6	60.1	**69.7**	63.6
	C^2TC	**69.6**	**70.3**	53.9	**64.7**	**61.5**	68.5	**64.8**
MI	Random	52.8	52.8	52.0	51.8	46.2	**52.7**	51.4
	DM	52.2	**53.2**	51.7	52.0	50.9	46.3	51.1
	MTT	53.4	52.0	52.1	**52.1**	49.8	52.5	52.0
	C^2TC	**53.6**	53.1	**52.2**	52.0	**51.5**	52.2	**52.4**

overall accuracy, whereas larger γ values emphasize minority-class clustering, leading to improved Macro-F1. Datasets with extreme imbalance ratios (e.g., JA, CO, and MI) exhibit distinct fluctuations, particularly in Macro-F1. This is because γ effectively reweights the allocation of representatives toward minority classes; under highly imbalanced settings, even small changes in γ can substantially affect minority-class representation. In practice, we suggest $\gamma = 0.25$ as a default setting, since it already yields strong overall performance without overly sacrificing Macro-F1. For scenarios with extreme class imbalance, we recommend setting $\gamma = 1$, as a larger γ places greater emphasis on minority classes and improves minority-class classification performance.

Step size scaling factor l. Figure A3(c) and (d) show the performance of C^2TC when varying $l \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$. We observe that multi-class datasets (JA, DA, CO, MI) are more sensitive to changes than binary ones (AD, AI). Specifically, larger-scale datasets such as CO and MI tend to benefit from larger values of l, while JA and DA peak around $l = 0.5$. This behavior can be attributed to the balance between exploration and exploitation: smaller l values accelerate step-size reduction, enabling finer adjustments in later iterations, while larger l values preserve broader exploration. In practice, we suggest $l = 0.5$ as a default value, as it offers a better balance between sufficient exploration in early iterations and stable exploitation in later stages. For large-scale multi-class datasets, moderately increasing the step size (l) can further enhance performance by preventing premature convergence.

F. Impact of Categorical Feature Imbalance

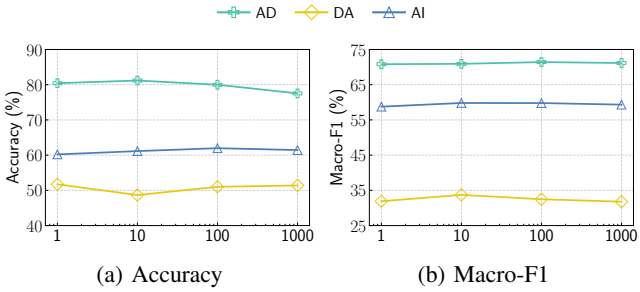
To evaluate the robustness of our categorical feature encoding method (HCFE) under severe feature imbalance commonly observed in real-world tabular datasets, we conduct experiments to examine the effects of varying imbalance levels on the proposed encoding and downstream model performance across three datasets with categorical features: AD, DA, and AI. Specifically, for each dataset, we select a categorical feature and construct training sets with controlled imbalance ratios of 1:1, 10:1, 100:1, and 1000:1 between two category values. For datasets containing a binary categorical feature, we directly use that column (e.g., sex in AD and diabetesMed in DA). For datasets without binary categorical features (e.g., AI), we select a multi-valued categorical column (DayOfWeek) and divide its seven unique values into two groups, assigning the first four days to 0 and the remaining three days to 1, yielding a binary feature on which controlled imbalance ratios can be imposed. These imbalance levels are achieved via a combination of label-conditioned downsampling and controlled upsampling on the selected feature. To ensure a fair and controlled comparison, all imbalance settings use the same number of training samples, strictly preserve the original label distribution, and share an identical test set, thereby isolating the effect of feature imbalance from other confounding factors. We evaluate downstream performance following the same condensation and learning pipeline using HCFE and the two baseline encoders. Figure A4 reports both accuracy and Macro-F1, where the x-axis denotes the feature



(a) γ Accuracy (b) γ Macro-F1

(c) l Accuracy (d) l Macro-F1

Fig. A3: Parameter sensitivity analysis of γ and l.



(a) Accuracy (b) Macro-F1

Fig. A4: Impact of categorical feature imbalance.

imbalance level (e.g., 1 corresponds to 1:1, 10 to 10:1). HCFE demonstrates consistently stable performance under different degrees of categorical feature imbalance on all three datasets. For example, on the AI dataset, the accuracy remains within a narrow range of 60.21%–61.97%, while Macro-F1 varies only between 58.76% and 59.81%. This limited performance fluctuation suggests that HCFE preserves stable representation quality even under extremely skewed categorical distributions.

REFERENCES

[1] M. Welling, "Herding dynamical weights to learn," in *ICML*, 2009, pp. 1121–1128.

[2] D. Feldman, M. Faulkner, and A. Krause, "Scalable training of mixture models via coresets," *NeurIPS*, vol. 24, 2011.

[3] O. Bachem, M. Lucic, and A. Krause, "Coresets for nonparametric estimation-the case of dp-means," in *ICML*, 2015, pp. 209–217.

[4] R. Z. Farahani and M. Hekmatfar, *Facility location: concepts, models, algorithms and case studies*. Springer Science & Business Media, 2009.

[5] V. Cohen-Addad, K. Green Larsen, D. Saulpic, C. Schwiegelshohn, and O. A. Sheikh-Omar, "Improved coresets for euclidean k-means," *NeurIPS*, vol. 35, pp. 2679–2694, 2022.

[6] B. Mirzasoleiman, J. Bilmes, and J. Leskovec, "Coresets for data-efficient training of machine learning models," in *ICML*, 2020, pp. 6950–6960.

[7] K. Killamsetty, S. Durga, G. Ramakrishnan, A. De, and R. Iyer, "Grad-match: Gradient matching based data subset selection for efficient deep model training," in *ICML*, 2021, pp. 5464–5474.

[8] M. Toneva, A. Sordoni, R. T. des Combes, A. Trischler, Y. Bengio, and G. J. Gordon, "An empirical study of example forgetting during deep neural network learning," in *ICLR*, 2019.

[9] A. Shrivastava, A. Gupta, and R. Girshick, "Training region-based object detectors with online hard example mining," in *CVPR*, 2016, pp. 761–769.

[10] A. Hadar, T. Milo, and K. Razmadze, "Datamap-driven tabular coreset selection for classifier training," *VLDB*, vol. 18, no. 3, pp. 876–888, 2024.

[11] B. Zhao and H. Bilen, "Dataset condensation with distribution matching," in *WACV*, 2023, pp. 6514–6523.

[12] G. Cazenavette, T. Wang, A. Torralba, A. A. Efros, and J.-Y. Zhu, "Dataset distillation by matching training trajectories," in *CVPR*, 2022, pp. 4750–4759.

[13] B. Zhao, K. R. Mopuri, and H. Bilen, "Dataset condensation with gradient matching," in *ICLR*, 2021.

[14] X. Gao, G. Ye, T. Chen, W. Zhang, J. Yu, and H. Yin, "Rethinking and accelerating graph condensation: A training-free approach with class partition," in *WWW*, 2025, pp. 4359–4373.

[15] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting deep learning models for tabular data," *NeurIPS*, vol. 34, pp. 18 932–18 943, 2021.

[16] J. Zar, "Biostatistical analysis," *Printice-Hall Inc*, 1999.

[17] V. Borisov, T. Leemann, K. Seßler, J. Haug, M. Pawelczyk, and G. Kasneci, "Deep neural networks and tabular data: A survey," *TNNLS*, vol. 35, no. 6, pp. 7499–7519, 2022.

[18] Y. Gorishniy, I. Rubachev, and A. Babenko, "On embeddings for numerical features in tabular deep learning," *NeurIPS*, vol. 35, pp. 24 991–25 004, 2022.

[19] D. Holzmüller, L. Grinsztajn, and I. Steinwart, "Better by default: Strong pre-tuned MLPs and boosted trees on tabular data," in *NeurIPS*, 2024.

[20] S. Ö. Arik and T. Pfister, "Tabnet: Attentive interpretable tabular learning," in *AAAI*, vol. 35, no. 8, 2021, pp. 6679–6687.

[21] Y. Gorishniy, I. Rubachev, N. Kartashev, D. Shlenskii, A. Kotelnikov, and A. Babenko, "Tabr: Tabular deep learning meets nearest neighbors," in *ICLR*, 2024.